Linear Regression with Python - Case Study

Problem Statement: You just got a new work contract with an E-commerce company based in New York City. The company sells clothing online but they also have in-store style and clothing advice sessions. Customers come into the store, have sessions/meetings with a personal stylist, then they can go home and order either on a mobile app or website for the clothes they want.

The company is trying to decide whether to focus their efforts on their mobile app experience or their website. They've hired you on contract to help them in figuring it out.

Objective: Create a model that allows to put in a few features concerning customers attitude and returns back an estimate of the yearly amount spent.

Import Data Analysis and Visualization Libraries

In [65]:
```
%pip install numpy pandas matplotlib seaborn openpyxl
%pip install ipywidgets

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
```

```
Requirement already satisfied: numpy in /usr/local/python/3.14.2/lib/python
3.14/site-packages (2.5.4)
Requirement already satisfied: pandas in /usr/local/python/3.14.2/lib/python
3.14/site-packages (3.1.0)
Requirement already satisfied: matplotlib in /usr/local/python/3.14.2/lib/py
thon3.14/site-packages (3.11.2)
Requirement already satisfied: seaborn in /usr/local/python/3.14.2/lib/pytho
n3.14/site-packages (0.13.2)
Requirement already satisfied: openpyxl in /usr/local/python/3.14.2/lib/pyth
on3.14/site-packages (3.1.5)
Requirement already satisfied: python-dateutil>=2.9.0 in /home/codespace/.lo
cal/lib/python3.14/site-packages (from pandas) (3.0.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/python/3.14.2/
lib/python3.14/site-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/python/3.14.2/lib/
python3.14/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in /usr/local/python/3.14.
2/lib/python3.14/site-packages (from matplotlib) (4.68.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/python/3.14.
2/lib/python3.14/site-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /home/codespace/.local/li
b/python3.14/site-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=9 in /usr/local/python/3.14.2/lib/pyt
hon3.14/site-packages (from matplotlib) (13.0.0)
Requirement already satisfied: pyparsing>=3 in /usr/local/python/3.14.2/lib/
python3.14/site-packages (from matplotlib) (3.3.3)
Requirement already satisfied: et-xmlfile in /usr/local/python/3.14.2/lib/py
thon3.14/site-packages (from openpyxl) (2.0.0)
Requirement already satisfied: six>=1.6 in /home/codespace/.local/lib/python
3.14/site-packages (from python-dateutil>=2.9.0->pandas) (1.17.0)
Requirement already satisfied: tzdata in /home/codespace/.local/lib/python3.
14/site-packages (from python-dateutil>=2.9.0->pandas) (2026.6)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: ipywidgets in /usr/local/python/3.14.2/lib/py
thon3.14/site-packages (8.1.9)
Requirement already satisfied: comm>=0.1.3 in /home/codespace/.local/lib/pyt
hon3.14/site-packages (from ipywidgets) (0.2.3)
Requirement already satisfied: ipython>=6.1.0 in /home/codespace/.local/lib/
python3.14/site-packages (from ipywidgets) (9.17.1)
Requirement already satisfied: traitlets>=4.3.1 in /home/codespace/.local/li
b/python3.14/site-packages (from ipywidgets) (5.16.1)
Requirement already satisfied: widgetsnbextension~=4.0.16 in /usr/local/pyth
on/3.14.2/lib/python3.14/site-packages (from ipywidgets) (4.0.16)
Requirement already satisfied: jupyterlab_widgets~=3.0.17 in /usr/local/pyth
on/3.14.2/lib/python3.14/site-packages (from ipywidgets) (3.0.17)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in /home/codes
pace/.local/lib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets)
(1.1.1)
Requirement already satisfied: jedi>=0.18.2 in /home/codespace/.local/lib/py
thon3.14/site-packages (from ipython>=6.1.0->ipywidgets) (0.20.1)
Requirement already satisfied: matplotlib-inline>=0.1.6 in /home/codespace/.
local/lib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets) (0.2.2)
Requirement already satisfied: pexpect>4.6 in /home/codespace/.local/lib/pyt
hon3.14/site-packages (from ipython>=6.1.0->ipywidgets) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in /home/codesp
ace/.local/lib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets)
```

```
(3.0.53)
Requirement already satisfied: psutil>=7 in /home/codespace/.local/lib/pytho
n3.14/site-packages (from ipython>=6.1.0->ipywidgets) (7.2.2)
Requirement already satisfied: pygments>=2.14.0 in /home/codespace/.local/li
b/python3.14/site-packages (from ipython>=6.1.0->ipywidgets) (2.21.0)
Requirement already satisfied: stack_data>=0.6.0 in /home/codespace/.local/l
ib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets) (0.6.3)
Requirement already satisfied: wcwidth>=0.1.4 in /home/codespace/.local/lib/
python3.14/site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=6.1.0
->ipywidgets) (0.9.4)
Requirement already satisfied: parso<0.9.0,>=0.8.7 in /home/codespace/.loca
l/lib/python3.14/site-packages (from jedi>=0.18.2->ipython>=6.1.0->ipywidget
s) (0.8.7)
Requirement already satisfied: ptyprocess>=0.5 in /home/codespace/.local/li
b/python3.14/site-packages (from pexpect>4.6->ipython>=6.1.0->ipywidgets)
(0.7.0)
Requirement already satisfied: executing>=1.2.0 in /home/codespace/.local/li
b/python3.14/site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidge
ts) (2.3.0)
Requirement already satisfied: asttokens>=2.1.0 in /home/codespace/.local/li
b/python3.14/site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidge
ts) (3.0.2)
Requirement already satisfied: pure-eval in /home/codespace/.local/lib/pytho
n3.14/site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (0.
2.4)
Note: you may need to restart the kernel to use updated packages.
```

Data Import

We'll work with the E-commerce Customers csv file from the company. It has Customer info, such as Email, Address, and their Avatar color. Then it also has numerical value columns:

- **Avg. Session Length**: Average length of in-store style advice sessions.
- **Time on App**: Average time spent on App in minutes.
- **Time on Website**: Average time spent on Website in minutes.
- **Length of Membership**: How many years the customer has been a member.

TO DO: Load the dataset and inspect its structure

In [66]:
```
customers = pd.read_csv("Ecommerce Customers.csv")
customers.head()
```

Out[66]:

	Email	Address	Avatar	Av Sessi Leng
0	mstephenson@fernandez.com	835 Frank Tunnel\nWrightmouth, MI 82180-9605	Violet	34.4972
1	hduke@hotmail.com	4547 Archer Common\nDiazchester, CA 06566-8576	DarkGreen	31.9262
2	pallen@yahoo.com	24645 Valerie Unions Suite 582\nCobbborough, D...	Bisque	33.0009
3	riverarebecca@gmail.com	1414 David Throughway\nPort Jason, OH 22070-1220	SaddleBrown	34.3055
4	mstephens@davidson-herman.com	14023 Rodriguez Passage\nPort Jacobville, PR 3...	MediumAquaMarine	33.3306

In [67]:
```
customers.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 515 entries, 0 to 514
Data columns (total 8 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   Email                 515 non-null    str    
 1   Address               515 non-null    str    
 2   Avatar                515 non-null    str    
 3   Avg. Session Length   515 non-null    float64
 4   Time on App           515 non-null    float64
 5   Time on Website       515 non-null    float64
 6   Length of Membership  515 non-null    float64
 7   Yearly Amount Spent   515 non-null    float64
dtypes: float64(5), str(3)
memory usage: 32.3 KB
```

Exploratory Data Analysis

Before generating models, it is best to take a look at the data we have available. Is there any trend we can recognize from the dataset? Are there any abnormal behaviors?

TO DO: Display correlations between feature

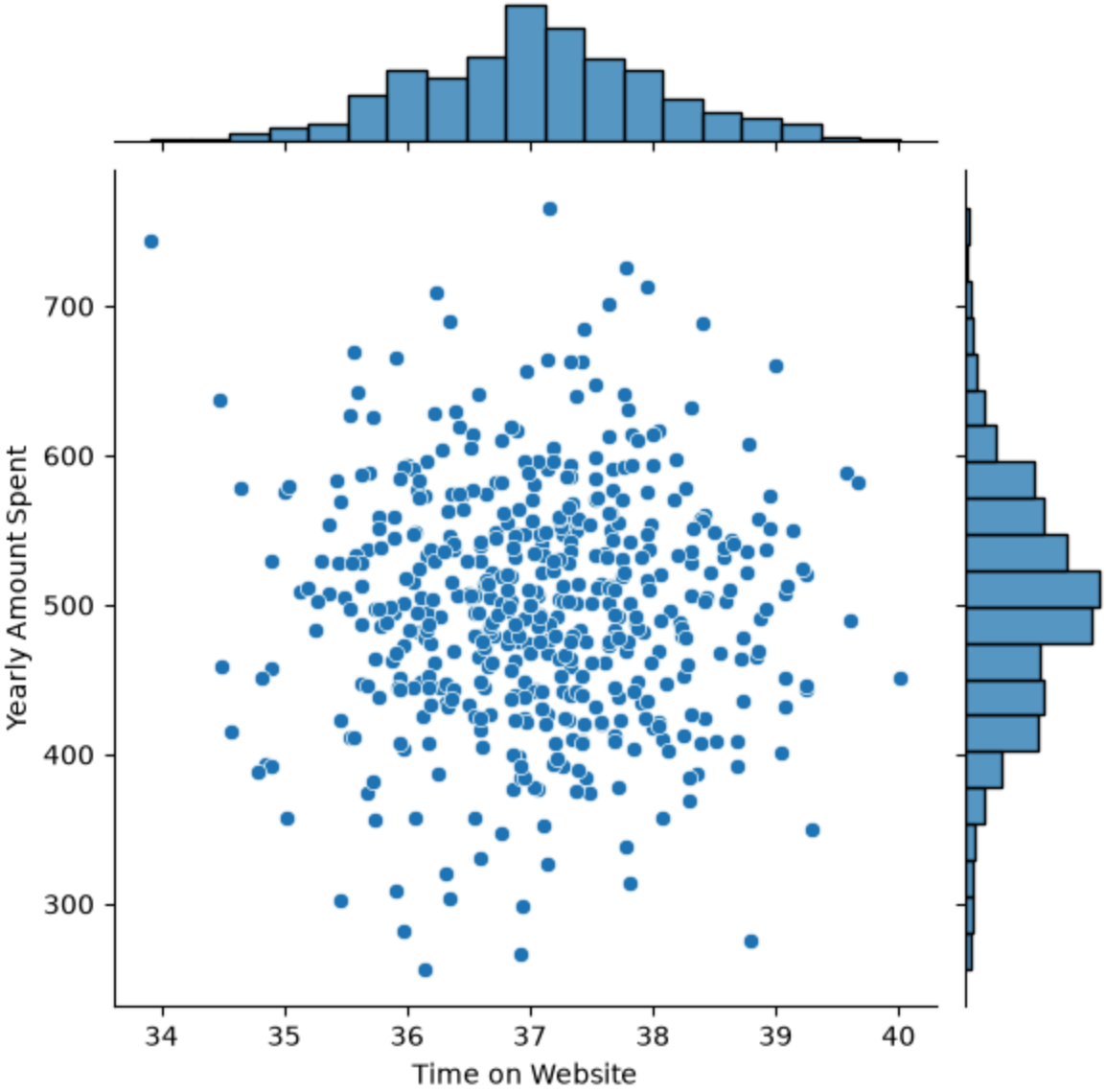
Focus on

1. Correlation between [Time on Website] and [Yearly Amount Spent]
2. Correlation across the entire dataset

Can you recognize some trends?

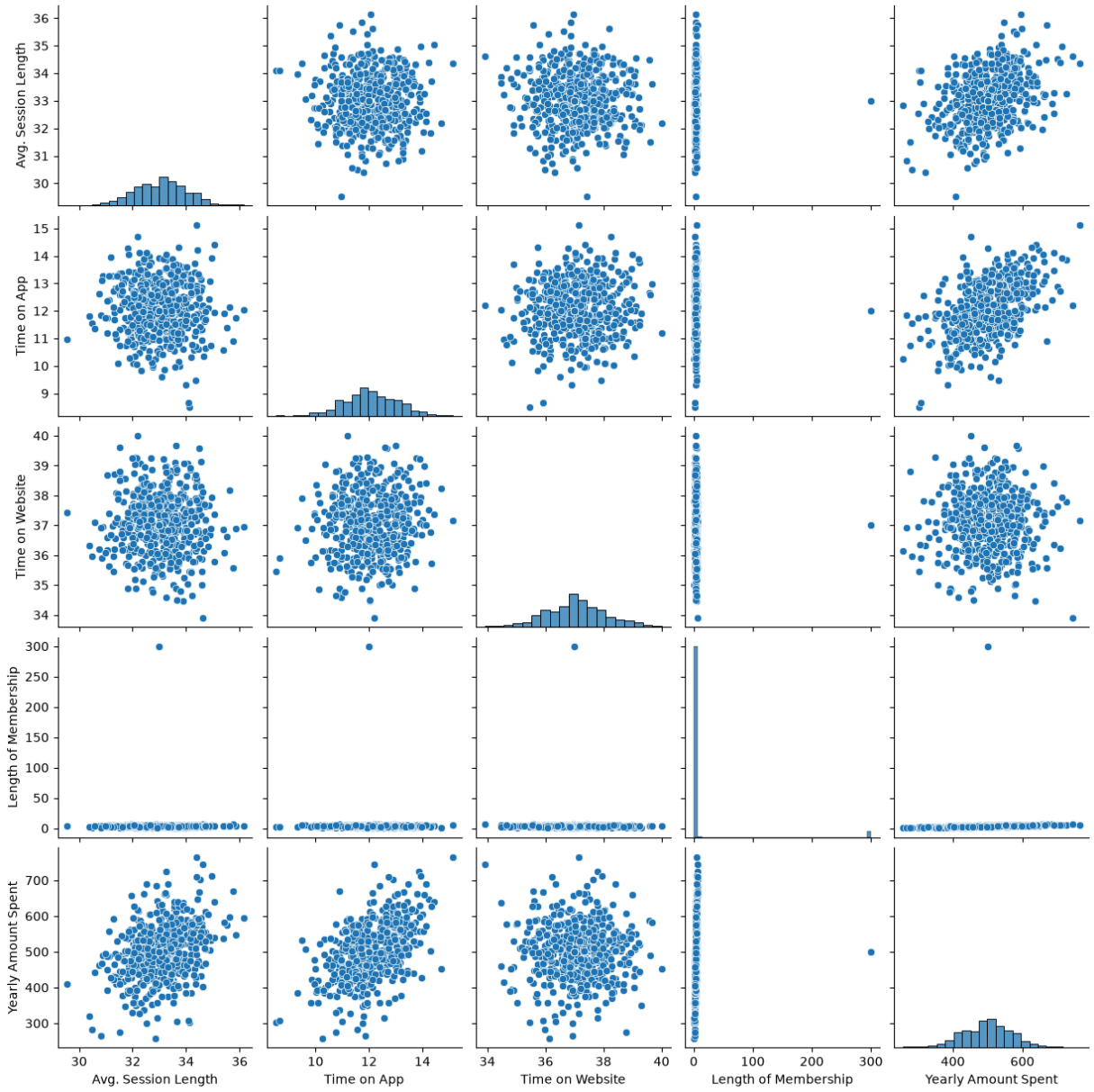
In [68]:
```
sns.jointplot(x='Time on Website', y='Yearly Amount Spent', data=customers)
```

Out[68]:
```
<seaborn.axisgrid.JointGrid at 0x76d004347c50>
```



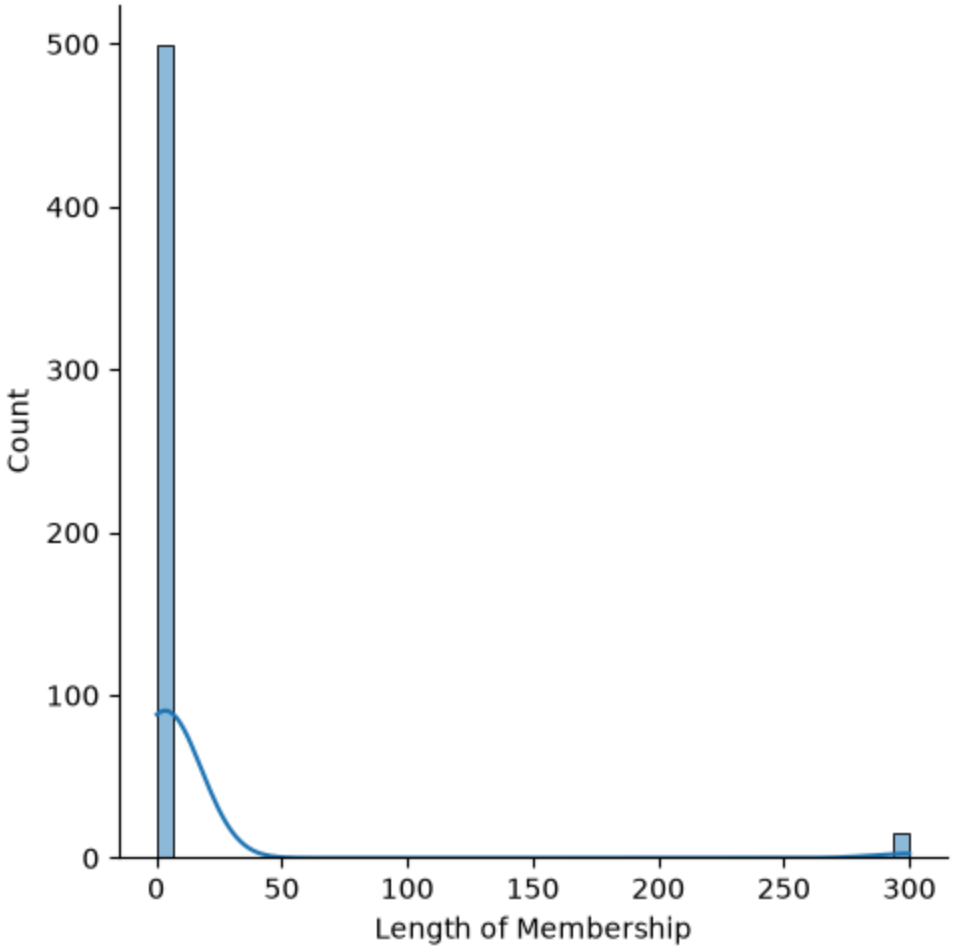
In [69]:
```
sns.pairplot(customers) # correlation of the all dataset
```

Out[69]:
```
<seaborn.axisgrid.PairGrid at 0x76d0073ee060>
```



In [70]:
```
sns.displot(customers['Length of Membership'], kde=True) # distribution
```

Out[70]:
```
<seaborn.axisgrid.FacetGrid at 0x76cffe93b890>
```



The membership length an outlier, which skews the data distribution! It is best to remove it before performing fitting any data, as it may deteriorate the performance of our model!

The IQR (Interquartile Range) method removes outliers by looking at how far values are from the middle of the data.

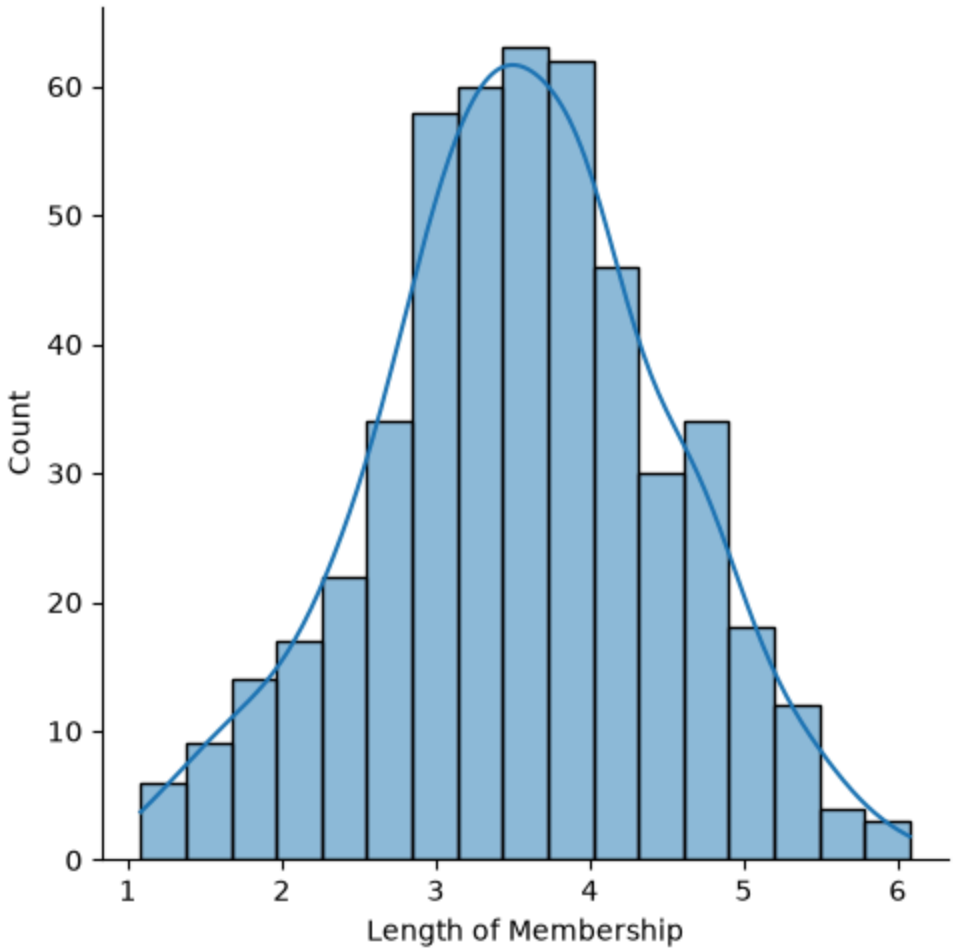
TO DO: Remove the outlier(s) and display the new data

In [71]:
```
Q1 = customers['Length of Membership'].quantile(0.25)
Q3 = customers['Length of Membership'].quantile(0.75)
IQR = Q3 - Q1

customers = customers[
    (customers['Length of Membership'] >= Q1 - 1.5*IQR) &
    (customers['Length of Membership'] <= Q3 + 1.5*IQR)
]

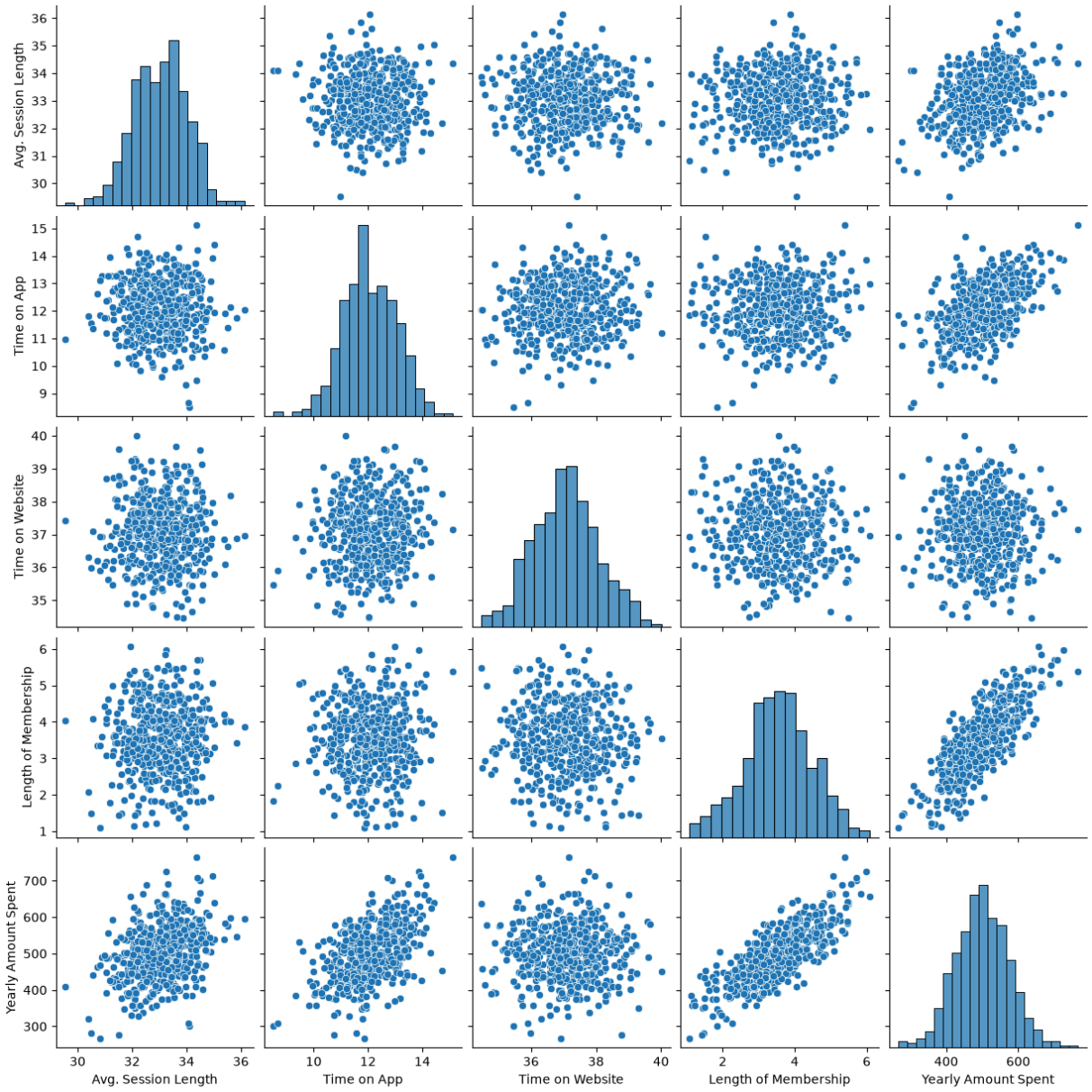
sns.displot(customers['Length of Membership'], kde=True)
```

Out[71]:
```
<seaborn.axisgrid.FacetGrid at 0x76cffe0c0cd0>
```



In [72]:
```
sns.pairplot(customers)
```

Out[72]:
```
<seaborn.axisgrid.PairGrid at 0x76d0069a84d0>
```



Evaluating correlation

In pandas, the .corr() method computes pairwise correlation coefficients between the columns of a DataFrame (or between two Series).

By default, it uses Pearson's correlation, which measures the linear relationship between two variables.

It returns values between -1 and 1:

- +1 → perfect positive linear correlation
- -1 → perfect negative linear correlation
- 0 → no linear correlation

You can also specify other methods:

- Kendall (rank correlation (monotonic, not strictly linear))
- Spearman (rank correlation (monotonic, based on ranks))

TO DO: Compute and display the correlation between the dataset feature

In [73]:
```
# customers.corr()
customers.drop(columns=["Email", "Address", "Avatar"]).corr()
```

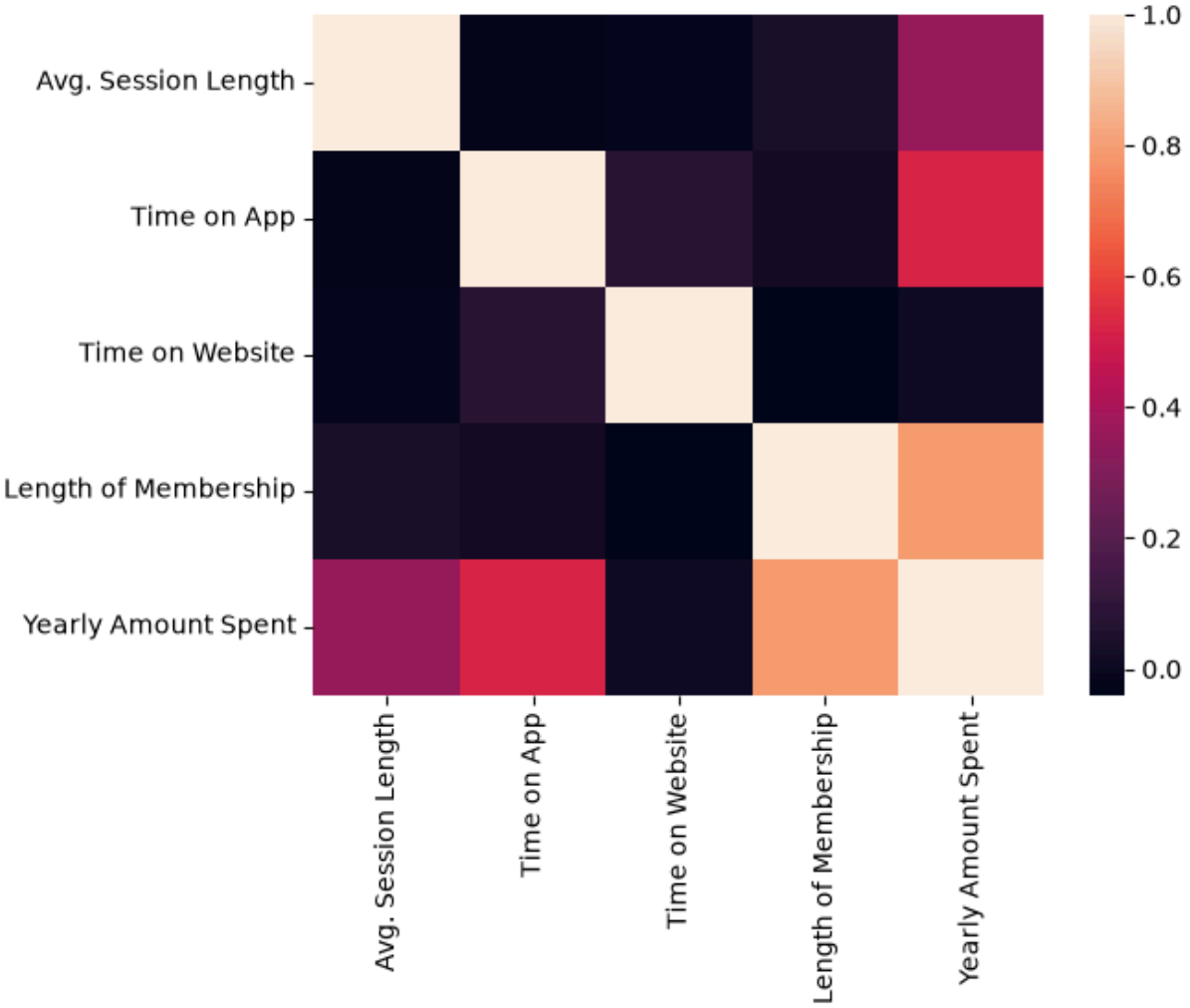
Out[73]:

	Avg. Session Length	Time on App	Time on Website	Length of Membership	Yearly Amount Spent
Avg. Session Length	1.000000	-0.022129	-0.014982	0.040285	0.356608
Time on App	-0.022129	1.000000	0.077265	0.021644	0.517277
Time on Website	-0.014982	0.077265	1.000000	-0.039242	0.008298
Length of Membership	0.040285	0.021644	-0.039242	1.000000	0.786708
Yearly Amount Spent	0.356608	0.517277	0.008298	0.786708	1.000000

In [74]:
```
correlation_matrix = customers.drop(columns=["Email", "Address", "Avatar"]).

# or
# numerical_columns = customers.select_dtypes(include=['number'])
# numerical_columns.head()
# correlation_matrix = numerical_columns.corr()
sns.heatmap(correlation_matrix)
```

Out[74]: <Axes: >



Based on these plots, what looks to be the most correlated feature with the Yearly Amount Spent?

Pearson correlation coefficient between two variables: This coefficient measures the strength and direction of the linear relationship between two continuous variables.

A small p-value (typically less than 0.05) suggests that the correlation is statistically significant, meaning there is evidence to reject the null hypothesis and conclude that a linear relationship likely exists.

For non-linear relationships, you can use: Spearman's rank correlation (ρ)

In [75]:
```
%pip install scipy
from scipy.stats import pearsonr
print('Membership - Membership:', pearsonr(customers['Length of Membership']
print('Amount spent - Membership:', pearsonr(customers['Yearly Amount Spent'
print('Amount spent - App:', pearsonr(customers['Time on App'], customers['Y
print('Amount spent - Website:', pearsonr(customers['Time on Website'], cust
```

```
Requirement already satisfied: scipy in /usr/local/python/3.14.2/lib/python
3.14/site-packages (1.18.1)
Requirement already satisfied: numpy<2.8,>=2.0.0 in /usr/local/python/3.14.
2/lib/python3.14/site-packages (from scipy) (2.5.4)
Note: you may need to restart the kernel to use updated packages.
Membership - Membership: PearsonRResult(statistic=np.float64(0.9999999999999
999), pvalue=np.float64(0.0))
Amount spent - Membership: PearsonRResult(statistic=np.float64(0.78670797561
57901), pvalue=np.float64(1.028423865491906e-104))
Amount spent - App: PearsonRResult(statistic=np.float64(0.5172766753006074),
pvalue=np.float64(5.098764572680568e-35))
Amount spent - Website: PearsonRResult(statistic=np.float64(0.00829830347517
5644), pvalue=np.float64(0.854327072140909))
```

Training a Linear Regression Model

We will need to first split up our data into an X array that contains the numerical features of the customers and a y array with the target variable, in this case the "Yearly Amount Spent" column.

We will toss out all the non-numerical columns because they only have text info that the linear regression model can't use.

In [76]:
```
X = customers[['Avg. Session Length', 'Time on App', 'Time on Website', 'Ler
y = customers['Yearly Amount Spent']
```

Train-Test Split (Supervised learning)

Now let's split the data into a training set and a test set. We will train our model on the training set and then use the test set to evaluate the model.

TO DO: Perform the train-test split.

In [77]:
```
%pip install scikit-learn
from sklearn.model_selection import train_test_split

# Manual splitting
X_train = X[:int(0.7*len(customers))]

# Function split. Balances also the output to avoid that we reshaped the dat
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, rar
```

```
Requirement already satisfied: scikit-learn in /usr/local/python/3.14.2/lib/
python3.14/site-packages (1.9.1)
Requirement already satisfied: numpy>=1.24.1 in /usr/local/python/3.14.2/li
b/python3.14/site-packages (from scikit-learn) (2.5.4)
Requirement already satisfied: scipy>=1.10.0 in /usr/local/python/3.14.2/li
b/python3.14/site-packages (from scikit-learn) (1.18.1)
Requirement already satisfied: joblib>=1.4.0 in /usr/local/python/3.14.2/li
b/python3.14/site-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: narwhals>=2.0.1 in /usr/local/python/3.14.2/l
ib/python3.14/site-packages (from scikit-learn) (2.28.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in /usr/local/python/3.1
4.2/lib/python3.14/site-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/python/3.14.2/
lib/python3.14/site-packages (from joblib>=1.4.0->scikit-learn) (3.1.2)
Note: you may need to restart the kernel to use updated packages.
```

train_test_split from sklearn shuffles the data by default before splitting. The parameter shuffle=True is set by default, ensuring that the data is randomized before being divided into training and testing sets.

- Avoid Bias: For instance, if the data is ordered by time
- Representative Samples: Shuffling ensures that both the training and test sets contain a good mix of samples
- Handling Class Imbalance: Ensuring that both sets have a balanced distribution of classes

random_state is the seed for reproducibility

Training the Model

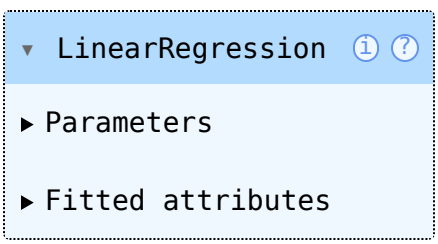
Now that we have divided between training and test data, we need to

1. Define a regression model
2. Train the regression model
3. Evaluate the performance of the regression model

TO DO: Instantiate and train a linear regression model (call it `lm`)

In [78]:
```
from sklearn.linear_model import LinearRegression
lm = LinearRegression()
lm.fit(X_train, y_train)
```

Out[78]:



In [79]:
```
lm.__dict__
```

Out[79]:
```
{'fit_intercept': True,
 'copy_X': True,
 'tol': 1e-06,
 'n_jobs': None,
 'positive': False,
 'feature_names_in_': array(['Avg. Session Length', 'Time on App', 'Time on
Website',
        'Length of Membership'], dtype=object),
 'n_features_in_': 4,
 'coef_': array([26.05405378, 38.72727531,  0.11940707, 61.16122379]),
 'rank_': np.int64(4),
 'singular_': array([19.08110461, 17.99406148, 17.60479746, 17.06210351]),
 'intercept_': np.float64(-1048.7353124986703)}
```

The linear model `lm` is an object with several attributes and methods (functions). To see the attributes, we can print `lm.__dict__` or `dir(lm)`.

TO DO: Display the linear model parameters

In [80]:
```
print('Bias:', lm.intercept_)
print('Coefficients:', lm.coef_)
```

```
Bias: -1048.7353124986703
Coefficients: [26.05405378 38.72727531  0.11940707 61.16122379]
```

Predicting Test Data and Regression Metrics

Now that we have trained a model, we can evaluate it on the test set. There are many existing evaluation metrics for regression, with some of the most common being:

Mean Absolute Error (MAE) is the mean of the absolute value of the errors:

$$\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Mean Squared Error (MSE) is the mean of the squared errors:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE) is the square root of the mean of the squared errors:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Comparing these metrics:

- **MAE** is the easiest to understand, because it's the average error, so it represents the error in the same units as the target variable..
- **MSE** is more popular than MAE, because MSE "punishes" larger errors, which could be useful in the real world. Sensitivity to Outliers: MSE is more sensitive to outliers because squaring the error magnifies the impact of larger deviations. Unit Issue: MSE is not in the same units as the target variable, but in its squared units, which can be less interpretable.
- **RMSE** is even more popular than MSE, because RMSE is interpretable in the "y" units. Sensitivity to Outliers: RMSE, like MSE, is sensitive to outliers due to squaring the errors, but because it takes the square root, the interpretation becomes clearer.

All of these are **loss functions**, because we want to minimize them.

TO DO: Compute, display, and evaluate the model predictions

In [81]:
```
from sklearn import metrics
predictions = lm.predict(X_test)
predictions[0]
```

Out[81]:
```
np.float64(551.2924451177798)
```

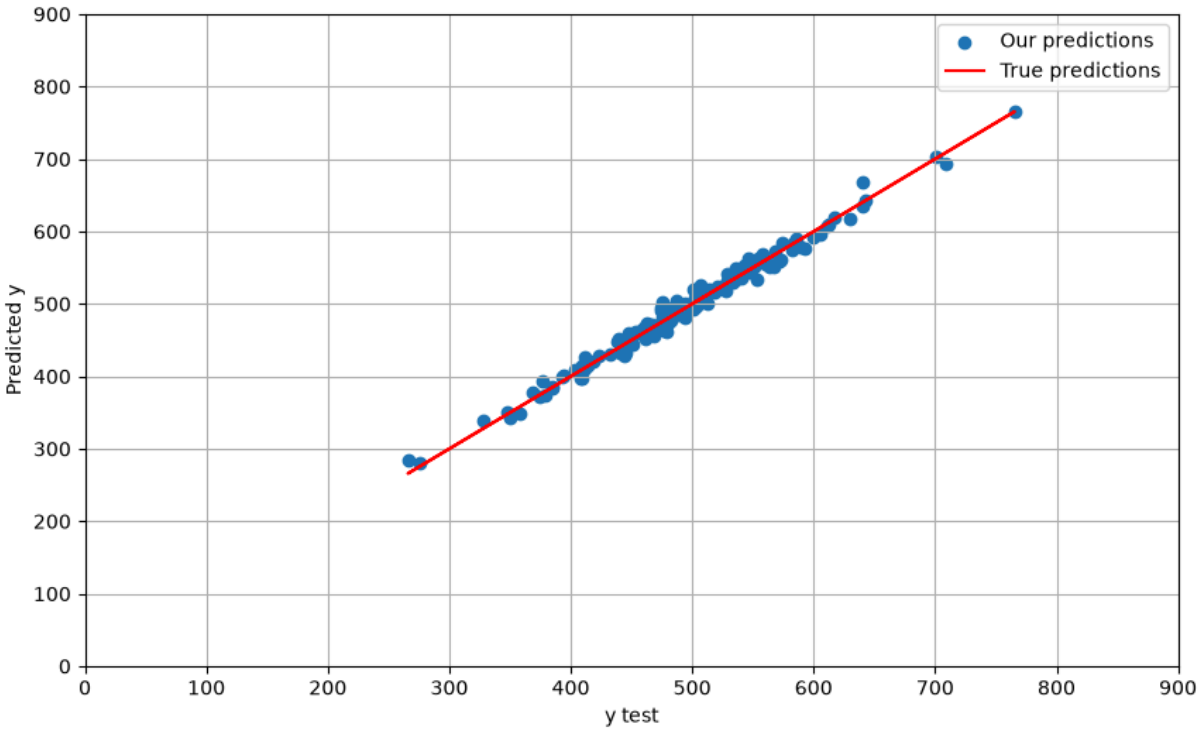
In [82]:
```
plt.figure(figsize=(10,6))

plt.scatter(y_test, predictions, label='Our predictions')
plt.plot(y_test, y_test, label="True predictions", color='red')

plt.xlabel('y test')
plt.ylabel('Predicted y')
plt.grid()
plt.xlim([0, 900])
plt.ylim([0, 900])
plt.legend()

plt.show
```

Out[82]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```



In [83]:
```
# performances of the model
print('MAE:', metrics.mean_absolute_error(y_test, predictions))
print('MSE:', metrics.mean_squared_error(y_test, predictions))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, predictions)))
```

```
MAE: 7.364711349957439
MSE: 84.82720279708323
RMSE: 9.210168445641113
```

Residuals

You should have obtained a very good model with a good fit. Let's quickly explore the residuals to make sure everything was fine with our data.

Plot a histogram of the residuals and make sure it looks normally distributed. If the linear regression model is appropriate and well-fitted, the residuals should follow a normal distribution.

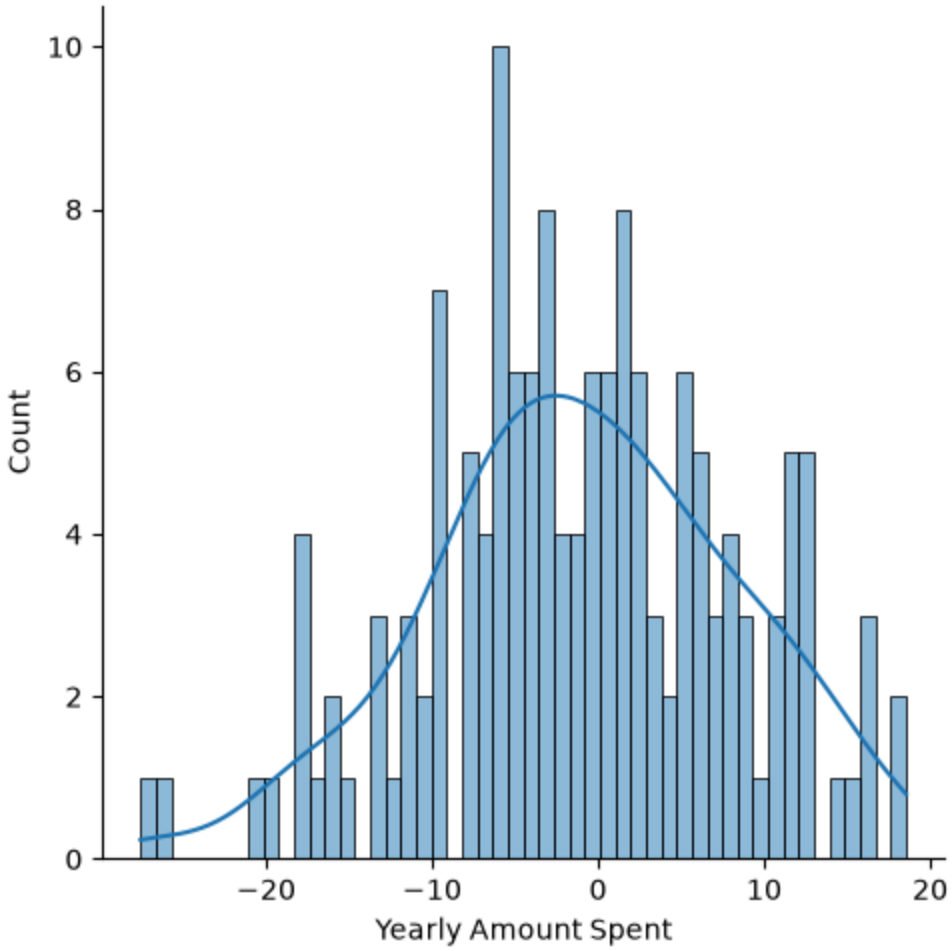
If Residuals Are Not Normally Distributed:

- Non-linearity: The relationship between the variables might not be linear. In that case, consider using polynomial regression or other non-linear models.
- Heteroscedasticity: If the variance of residuals changes across the dataset, this violates the homoscedasticity assumption. You may need to transform the data or use weighted regression.
- Outliers: Extreme values could affect normality. Removing or addressing outliers might improve the model fit.

TO DO: Display the prediction residuals

In [84]:
```
sns.displot( (y_test-predictions), bins=50, kde=True)
# for a good model the residual should be gaussian
```

Out[84]: <seaborn.axisgrid.FacetGrid at 0x76cffcf1dbd0>



Conclusion

We still want to figure out the answer to the original question, do we focus our effort on mobile app or website development? Or maybe that doesn't even really matter, and Membership Time is what is really important. Let's see if we can interpret the coefficients to get an idea.

In [85]:
```
coefficients = pd.DataFrame(lm.coef_,X.columns)
coefficients.columns = ['Coefficient']
coefficients
```

Out[85]:

	Coefficient
Avg. Session Length	26.054054
Time on App	38.727275
Time on Website	0.119407
Length of Membership	61.161224

Interpreting the coefficients:

- Holding all other features fixed, a 1 unit increase in **Avg. Session Length** is associated with an **increase of 26.05 total dollars spent**.
- Holding all other features fixed, a 1 unit increase in **Time on App** is associated with an **increase of 38.72 total dollars spent**.
- Holding all other features fixed, a 1 unit increase in **Time on Website** is associated with an **increase of 0.11 total dollars spent**.
- Holding all other features fixed, a 1 unit increase in **Length of Membership** is associated with an **increase of 61.16 total dollars spent**.

Do you think the company should focus more on their mobile app or on their website?

This is tricky, there are two ways to think about this: develop the website to catch up to the performance of the mobile app, or develop the app more since that is what is working better. This sort of answer really depends on the other factors going on at the company, you would probably want to explore the relationship between Length of Membership and the App or the Website before coming to a conclusion!

Great Job!

Congrats on your contract! The company loved the insights!